

EECE 7205: Fundamentals of Computer Engineering
Fall 2026
Assignment #01: Sorting Applications and Algorithm Analysis

Student Name: Natan Blumberg
NUID: 003984173
Email: blumberg.n@northeastern.edu
Instructor: Dr. Fatema Nafa
Due Date: September 24th, 2026

This assignment was completed with the help of AI. I used AI to:

- Question me about faults in my pseudocode.
- How best to format my C++ code to make the "main menu" functionality work.
- Teach me some concepts in C++.
- Identify errors and bugs in my code.
- Assist in writing the LaTeX for this assignment. (Tables, equations, etc.)

AI did NOT:

- Write my code (Pseudocode or C++)
- Do any thinking or problem solving that impacted methods on solving these problems.

1 Merge Overlapping Intervals

1.1 Understand the Problem

Input: An array of intervals [start, end], in no particular order.

Output: The smallest set of non-overlapping intervals that covers the same ranges, sorted by start value.

Assumptions: start is always less than or equal to end for every interval. Intervals that touch at an endpoint (for example [1,4] and [4,5]) are treated as overlapping.

Edge Cases:

- Empty input - return empty output
- Single interval - return it unchanged
- No intervals overlap - output equals the sorted input
- All intervals overlap into one - output has exactly one interval
- One interval fully contained inside another (for example [1,10] and [2,3])

1.2 Design the Solution

Sorting the intervals by start value turns this into a single left-to-right scan. Once sorted, any interval that could overlap the current one must be the most recently merged interval, so only one comparison is needed per element instead of comparing every pair.

1.3 Pseudocode

Algorithm: Merge Overlapping Intervals

Input: An array of intervals, Intervals, each with a start and an end

Output: Result, the smallest set of non-overlapping intervals

```
1. n <- SizeOf(Intervals)
2. if n = 0 then
3.     return empty array
4. end if
5. Sort(Intervals) by increasing start value
6. Result <- empty array
7. Append(Result, Intervals[0])
8. for i <- 1 to n - 1 do
9.     LastIndex <- SizeOf(Result) - 1
10.    if Intervals[i].start <= Result[LastIndex].end then
11.        if Intervals[i].end > Result[LastIndex].end then
12.            Result[LastIndex].end <- Intervals[i].end
13.        end if
14.    else
15.        Append(Result, Intervals[i])
16.    end if
17. end for
18. return Result
```

1.4 Correctness

At every point in the loop, Result holds the correct merged, non-overlapping set of all intervals processed so far. Since the array is sorted by start value, any interval already placed in Result (other than the last one) can never be reached again, so the current interval can only possibly overlap Result's last entry. Taking the larger of the two end values (line 12) handles the case where a later interval is fully contained inside an earlier one. Empty input and single-interval cases are handled independently.

1.5 Complexity Analysis

Cost model: every assignment, comparison, and append costs 1 unit. The Sort on line 5 is merge sort. It halves the array and does about cn work per level to merge, so

$$T_{\text{sort}}(n) = 2T_{\text{sort}}(n/2) + cn = cn \log_2 n + cn$$

The loop on lines 8–17 costs $2n$ for loop control and runs $n - 1$ times. An overlapping interval costs 4 units (lines 9–12) and a separate one costs 3 (lines 9, 10, 15). Lines 1, 2, 6, 7, and 18 add 5.

$$T_{\text{worst}}(n) = T_{\text{sort}}(n) + 2n + 4(n - 1) + 5 = cn \log_2 n + cn + 6n + 1$$

$$T_{\text{best}}(n) = T_{\text{sort}}(n) + 2n + 3(n - 1) + 5 = cn \log_2 n + cn + 5n + 2$$

Merge sort does the same work on every input, so the sort dominates in every case. Space is $\Theta(n)$ for the halves merge sort copies and Result.

Best	Average	Worst	Aux. Space
$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$

1.6 Testing

Test	Input	Expected Output	Edge Case?
1	[1,3] [2,6] [8,10] [15,18]	[1,6] [8,10] [15,18]	No
2	[1,4] [4,5]	[1,5]	Yes (touching endpoints)
3	(empty array)	(empty array)	Yes (empty input)

Execution screenshots for these three tests are in the Appendix, Problem 1.

2 Sort Array of 0s, 1s, and 2s

2.1 Understand the Problem

Input: An array containing only the values 0, 1, and 2.

Output: The same array, rearranged in place so all 0s come first, then all 1s, then all 2s.

Assumptions: Every element is exactly 0, 1, or 2. Sorting is done in place.

Edge Cases:

- Empty array
- Array of all one value
- Already sorted array
- Reverse sorted array (for example 2,2,1,1,0,0)

2.2 Design the Solution

Two solutions are required. First, a counting-based approach: count how many 0s, 1s, and 2s exist, then overwrite the array using those counts. Second, the Dutch National Flag algorithm: use three pointers (low, mid, high) to sort the array in a single pass.

2.3 Pseudocode

Algorithm: Counting Sort for 0s, 1s, and 2s

Input: An array Nums, where every value is 0, 1, or 2

Output: Nums, rearranged so all 0s come first, then all 1s, then all 2s

```
1. n <- SizeOf(Nums)
2. CountZero <- 0
3. CountOne <- 0
4. CountTwo <- 0
5. for i <- 0 to n - 1 do
6.     if Nums[i] = 0 then
7.         CountZero <- CountZero + 1
8.     else if Nums[i] = 1 then
9.         CountOne <- CountOne + 1
10.    else
11.        CountTwo <- CountTwo + 1
12.    end if
13. end for
14. Index <- 0
15. write CountZero copies of 0 into Nums, starting at Index, then update Index
16. write CountOne copies of 1 into Nums, starting at Index, then update Index
17. write CountTwo copies of 2 into Nums, starting at Index, then update Index
18. return Nums
```

Algorithm: Dutch National Flag Partition (One Pass)

Input: An array Nums, where every value is 0, 1, or 2

Output: Nums, rearranged in place so all 0s come first, then all 1s, then all 2s

```
1. n <- SizeOf(Nums)
2. Low <- 0
```

```

3. Mid <- 0
4. High <- n - 1
5. while Mid <= High do
6.     if Nums[Mid] = 0 then
7.         Swap(Nums[Low], Nums[Mid])
8.         Low <- Low + 1
9.         Mid <- Mid + 1
10.    else if Nums[Mid] = 1 then
11.        Mid <- Mid + 1
12.    else
13.        Swap(Nums[Mid], Nums[High])
14.        High <- High - 1
15.    end if
16. end while
17. return Nums

```

2.4 Correctness

Counting-based: correctness follows directly from the definition of counting sort restricted to three values. The counts guarantee the exact right number of each value is written back in order.

Dutch Flag: at every step, the array is divided into four regions: positions before Low are all 0s, positions from Low to Mid-1 are all 1s, positions from Mid to High are unknown, and positions after High are all 2s. Each branch of the loop correctly shrinks the unknown region by one element. The loop ends when Mid is greater than High, meaning every element has been correctly classified.

2.5 Complexity Analysis

Cost model: every assignment, comparison, increment, and swap costs 1 unit.

Counting-based: loop control on line 5 costs $2n + 2$. Each iteration runs the test on line 6 plus one increment (a 0), or the test on line 8 plus one increment (a 1 or 2). Lines 15–17 do n writes and n Index updates, and lines 1–4, 14, and 18 add 6.

$$T_{worst}(n) = (2n + 2) + 3n + 2n + 6 = 7n + 8 \quad T_{best}(n) = 6n + 8 \text{ (all 0s)}$$

Dutch National Flag: each iteration moves Mid up or High down by one, so the body runs exactly n times and the test on line 5 runs $n + 1$ times. A 0 or 2 costs 4 units and a 1 costs 3. Lines 1–4 and 17 add 5.

$$T_{worst}(n) = (n + 1) + 4n + 5 = 5n + 6 \quad T_{best}(n) = 4n + 6 \text{ (all 1s)}$$

These are exact counts, so the Θ bounds are tight.

Approach	Best	Average	Worst	Aux. Space
Counting	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Dutch Flag	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$

2.6 Testing

Test	Input	Expected Output	Edge Case?
1	2 0 2 1 1 0	0 0 1 1 2 2	No
2	0 0 1 1 2 2	0 0 1 1 2 2	Yes (already sorted)
3	(empty array)	(empty array)	Yes (empty input)
4	1 1 1 1	1 1 1 1	Yes (all one value)

Execution screenshots for these tests are in the Appendix, Problem 2.

3 Largest Number from Concatenation

3.1 Understand the Problem

Input: A vector of nonnegative integers.

Output: A string representing the largest number formed by concatenating all integers in some order.

Assumptions: All integers are nonnegative. The result must be a string, since the number can be too large for a normal numeric type.

Edge Cases:

- Empty array - return "0" (stated assumption)
- Single number - return it unchanged as a string
- All zeros (for example [0,0]) - must return "0", not "00"
- Numbers with equal digits but different order (for example [3,30])

3.2 Design the Solution

Convert every number to a string, then sort using a comparator: string A comes before string B if A+B (as a concatenated string) is numerically larger than B+A. This directly compares what happens if a number goes first, without needing to reason about digit lengths separately.

3.3 Pseudocode

Algorithm: Largest Number by Concatenation

Input: An array of nonnegative integers, Nums

Output: LargestNumber, the largest number formed by concatenating all values

```
1. if SizeOf(Nums) = 0 then
2.     return "0"
3. end if
4. Strings <- empty array of strings
5. for each value in Nums do
6.     Append(Strings, ToString(value))
7. end for
8. Sort(Strings) using this rule:
   A comes before B if Concatenate(A,B) >= Concatenate(B,A)
9. Result <- ""
10. for each s in Strings do
11.     Result <- Concatenate(Result, s)
12. end for
13. if Result[0] = '0' then
14.     return "0"
15. end if
16. return Result
```

3.4 Correctness

Suppose in an optimal arrangement, number A immediately precedes B, but Concatenate(B,A) is larger than Concatenate(A,B). Swapping them would produce a larger number, which contradicts optimality. So in any optimal arrangement, every adjacent pair must satisfy Concatenate(A,B) is greater than or equal to Concatenate(B,A). The all-zero case is handled explicitly: if the first character of the result is '0', every digit must be '0', so the whole result collapses to "0".

3.5 Complexity Analysis

Let n be the number of integers and k the maximum number of digits. Lines 4–7 convert every number, costing about c_1nk . The Sort on line 8 is merge sort, but each comparison builds A+B and B+A, costing up to c_2k :

$$T_{\text{sort}}(n) = 2T_{\text{sort}}(n/2) + c_2kn = c_2kn \log_2 n + c_2kn$$

Lines 10–12 append n strings of at most k digits in place, costing at most nk .

$$T(n) = c_2kn \log_2 n + (c_1 + c_2 + 1)nk + c_3 = \Theta(kn \log n)$$

Merge sort does the same work on every input, so all cases match. Space is $\Theta(nk)$ for the strings and the result.

Best	Average	Worst	Aux. Space
$\Theta(kn \log n)$	$\Theta(kn \log n)$	$\Theta(kn \log n)$	$\Theta(nk)$

3.6 Testing

Test	Input	Expected Output	Edge Case?
1	3 30 34 5 9	"9534330"	No
2	0 0	"0"	Yes (all zeros)
3	5	"5"	Yes (single value)

Execution screenshots for these tests are in the Appendix, Problem 3.

4 Group Anagrams

4.1 Understand the Problem

Input: A vector of strings.

Output: The strings grouped so anagrams of each other are together. Group order does not matter, but words inside a group must keep their original relative order.

Assumptions: Comparison is case-sensitive unless stated otherwise.

Edge Cases:

- Empty input
- No two words are anagrams (every group has one word)
- All words identical
- Empty strings, which are all anagrams of each other

4.2 Design the Solution

Two words are anagrams exactly when sorting their letters produces identical strings. Using this sorted form as a hash map key groups all anagrams together in a single pass, instead of comparing every pair of words.

4.3 Pseudocode

Algorithm: Group Anagrams

Input: An array of strings, Words

Output: Groups, lists of anagrams, keeping original order within each group

```
1. HashMap <- empty map from string to list of strings
2. for each word in Words, in order do
3.     Key <- SortLetters(word)
4.     Append(HashMap[Key], word)
5. end for
6. Groups <- empty list
7. for each Key in HashMap do
8.     Append(Groups, HashMap[Key])
9. end for
10. return Groups
```

4.4 Correctness

SortLetters produces identical output for two strings if and only if they are anagrams, so it is a valid key for grouping. Since words are processed in their original order and appended to their group as they are found, each group naturally preserves original relative order. Empty strings all sort to "" and correctly group together. Identical words are trivially anagrams of themselves.

4.5 Complexity Analysis

Let n be the number of words, k the maximum word length, and g the number of groups. For each word, SortLetters is a merge sort on k letters ($ck \log_2 k + ck$), hashing the key (adding up its letter codes, mod 101) costs k , and the lookup compares the key with the other keys in its bucket (up to k each). With 101 buckets and keys spread evenly, a bucket holds about $g/101$ keys:

$$T(n) = n(ck \log_2 k + ck) + nk + nk \cdot \frac{g}{101}$$

Best case is one group ($g = 1$). Worst case is every word in its own group with every key landing in the same bucket ($g = n$, all in one chain). For small inputs like these, g is far below 101, so the lookup is effectively constant. Space is $\Theta(nk)$ for the groups and keys.

Best	Average	Worst	Aux. Space
$\Theta(nk \log k)$	$\Theta(nk \log k + nk g / 101)$	$\Theta(nk \log k + n^2 k)$	$\Theta(nk)$

4.6 Testing

Test	Input	Expected Output	Edge Case?
1	eat tea tan ate nat bat	{eat,tea,ate} {tan,nat} {bat}	No
2	two empty strings	one group with both	Yes (empty strings)
3	abc def ghi	three separate groups	Yes (no anagrams)

Execution screenshots for these tests are in the Appendix, Problem 4.

5 Meeting Rooms (Can Attend All?)

5.1 Understand the Problem

Input: An array of meeting intervals [start, end].

Output: true if one person can attend every meeting with no overlaps, false otherwise.

Assumption: A meeting starting exactly when another ends is allowed.

Edge Cases:

- Empty input - true
- Single meeting - true
- Meetings touching at endpoints - true
- Overlapping meetings - false

5.2 Design the Solution

Sort by start time, then check only neighboring meetings in a single pass. If sorted meetings never conflict with the meeting right before them, no conflict exists anywhere in the array.

5.3 Pseudocode

Algorithm: Check If One Person Can Attend All Meetings

Input: An array of meeting intervals, Meetings

Output: true if no two meetings overlap, false otherwise

```
1. n <- SizeOf(Meetings)
2. if n <= 1 then
3.     return true
4. end if
5. Sort(Meetings) by increasing start value
6. for i <- 1 to n - 1 do
7.     if Meetings[i].start < Meetings[i-1].end then
8.         return false
9.     end if
10. end for
11. return true
```

5.4 Correctness

Suppose, for contradiction, that two meetings that are not next to each other overlap. Because the array is sorted by start time, the meeting right between them must also overlap with at least one of them. This means an adjacent pair would also have failed the check, which contradicts the assumption that all adjacent pairs passed. So checking only neighboring pairs is enough. Using strict less-than (not less-than-or-equal) on line 7 makes sure touching meetings are allowed.

5.5 Complexity Analysis

Cost model: every assignment and comparison costs 1 unit. The Sort on line 5 is merge sort, $T_{sort}(n) = cn \log_2 n + cn$ (derived in Problem 1). The scan on lines 6–10 costs $2n$ for loop control plus one test per iteration, and lines 1, 2, and 11 add 3.

$$T_{worst}(n) = T_{sort}(n) + 2n + (n - 1) + 3 = cn \log_2 n + cn + 3n + 2$$

$$T_{best}(n) = T_{sort}(n) + c_0 \text{ (conflict at the first pair)}$$

The early exit only shortens the scan. The sort always runs in full, so every case is $\Theta(n \log n)$. Space is $\Theta(n)$ for the copies merge sort makes.

Best	Average	Worst	Aux. Space
$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$

5.6 Testing

Test	Input	Expected Output	Edge Case?
1	[0,30] [5,10] [15,20]	false	No
2	[1,4] [4,5]	true	Yes (touching endpoints)
3	(empty array)	true	Yes (empty input)

Execution screenshots for these tests are in the Appendix, Problem 5.

6 K Closest Points to Origin

6.1 Understand the Problem

Input: An array of points (x, y) and an integer k.

Output: The k points closest to the origin. Order within the result does not matter.

Assumption: Squared distance (x squared plus y squared) is used instead of true distance, since it gives the same ordering without needing a square root.

Edge Cases:

- k = 0 - empty result
- k = n - return all points
- k greater than n - invalid, must be flagged
- Points tied in distance - any valid combination is acceptable

6.2 Design the Solution

Two approaches are required. First, sort all points by distance and take the first k ($O(n \log n)$). Second, keep a max-heap of size k, so the heap always holds the k best candidates seen so far, replacing the current farthest whenever a closer point is found ($O(n \log k)$, which is better when k is much smaller than n).

6.3 Pseudocode

Algorithm: K Closest Points - Sorting Approach

Input: An array of points, Points, and an integer k

Output: Closest, the k points nearest the origin

```
1. if k <= 0 then
2.     return empty array
3. end if
4. for each point in Points do
5.     point.distance <- point.x * point.x + point.y * point.y
6. end for
7. Sort(Points) by increasing distance
8. Closest <- first k elements of Points
9. return Closest
```

Algorithm: K Closest Points - Max-Heap Approach

Input: An array of points, Points, and an integer k

Output: Closest, the k points nearest the origin

```
1. if k <= 0 then
2.     return empty array
3. end if
4. MaxHeap <- empty max-heap ordered by distance
5. for each point in Points do
6.     d <- point.x * point.x + point.y * point.y
7.     if SizeOf(MaxHeap) < k then
8.         Insert(MaxHeap, point, d)
9.     else if d < PeekMax(MaxHeap).distance then
10.        RemoveMax(MaxHeap)
11.        Insert(MaxHeap, point, d)
```

```

12.     end if
13. end for
14. Closest <- all elements in MaxHeap
15. return Closest

```

6.4 Correctness

Sorting approach: sorting by distance places the smallest distances first by definition, so the first k are exactly the k closest.

Max-heap approach: the heap always holds the k smallest-distance points seen so far. Once full, if a new point is closer than the heap's current farthest, that farthest point cannot be among the true k closest overall, so it is safely removed. The check on line 1 prevents looking at an empty heap when k is 0.

6.5 Complexity Analysis

Sorting: lines 4–6 compute n distances at 4 units each (two multiplications, one addition, one assignment), line 7 is merge sort, and line 8 copies k points:

$$T(n) = cn \log_2 n + cn + 4n + k + c_0 = \Theta(n \log n)$$

Max-heap: each point costs c_1 for its distance and tests. An insert or a replace moves a point at most $\lceil \log_2 k \rceil$ levels.

$$T_{worst}(n) \leq c_1 n + c_2 n \log_2 k \quad T_{best}(n) = c_1 n$$

The worst case is when every point enters the heap and moves the full height. The best case is when the first k points arrive farthest-first (no sifting) and every later point is farther than the heap maximum.

Approach	Best	Average	Worst	Aux. Space
Sorting	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
Max-Heap	$\Theta(n)$	$O(n \log k)$	$\Theta(n \log k)$	$\Theta(k)$

The heap approach wins when k is much smaller than n .

6.6 Testing

Test	Input	Expected Output	Edge Case?
1	(1,3)(-2,2)(5,8)(0,1), $k=2$	(0,1),(-2,2)	No
2	(1,1)(2,2), $k=2$	both points	Yes ($k = n$)
3	(1,1)(2,2), $k=0$	empty result	Yes ($k = 0$)

Execution screenshots for these tests are in the Appendix, Problem 6.

7 Top K Frequent Elements

7.1 Understand the Problem

Input: An integer array `Nums` and an integer `k`.

Output: The `k` values with the highest frequency. If frequencies tie, the smaller value comes first.

Edge Cases:

- Empty array
- `k` equals the number of distinct values
- All elements distinct (all tied at frequency 1)
- `k` greater than the number of distinct values - invalid, must be flagged

7.2 Design the Solution

Count the frequency of each value with a hash map, then sort the distinct values with a rule that orders by frequency from high to low, and breaks ties by choosing the smaller value first. Take the first `k` values.

7.3 Pseudocode

Algorithm: Top K Frequent Elements

Input: An array of integers, `Nums`, and an integer `k`

Output: `TopK`, the `k` values with highest frequency (ties: smaller value first)

```
1. if k <= 0 then
2.     return empty array
3. end if
4. FreqMap <- empty map from value to count
5. for each value in Nums do
6.     FreqMap[value] <- FreqMap[value] + 1
7. end for
8. DistinctValues <- all keys of FreqMap
9. if k > SizeOf(DistinctValues) then
10.    report invalid input and stop
11. end if
12. Sort(DistinctValues) using this rule:
    A comes before B if FreqMap[A] > FreqMap[B],
    or if they are tied and A < B
13. TopK <- first k elements of DistinctValues
14. return TopK
```

7.4 Correctness

The sorting rule defines a clear order: frequency from high to low, then value from low to high as a tie-break. Sorting by this rule places, at every point in the list, exactly the values that most deserve to be included. So taking the first `k` values is correct even when the cutoff falls in the middle of a group of tied frequencies.

7.5 Complexity Analysis

Let n be the array length and d the number of distinct values. Lines 5–7 look up each value in its bucket (value mod 101). With values spread evenly, a bucket holds about $d/101$ entries. Line 12 is merge sort on d keys, and line 13 copies k values:

$$T(n) = c_1n + c_2n \cdot \frac{d}{101} + c_3d \log_2 d + c_3d + k$$

The best case is when every value is the same ($d = 1$). The worst case is when every value is distinct and all values land in one bucket (for example, all multiples of 101), so each lookup scans every earlier value. When d is small compared to 101 buckets, the average is effectively $\Theta(n + d \log d)$. Space is $\Theta(d)$.

Best	Average	Worst	Aux. Space
$\Theta(n)$	$\Theta(nd/101 + d \log d)$	$\Theta(n^2)$	$\Theta(d)$

7.6 Testing

Test	Input	Expected Output	Edge Case?
1	1 1 1 2 2 3, k=2	1 2	No
2	3 1 2, k=2	1 2	Yes (all tied)
3	5 5 6, k=2	5 6	Yes (k = distinct count)

Execution screenshots for these tests are in the Appendix, Problem 7.

8 Merge K Sorted Arrays

8.1 Understand the Problem

Input: k individually sorted integer arrays.

Output: One sorted array containing every element from every input array.

Requirement: must not simply put everything into one array and re-sort the whole thing.

Edge Cases:

- k = 0 (no arrays)
- Some arrays empty
- All arrays empty
- k = 1

8.2 Design the Solution

Keep a min-heap holding one current candidate from each array. Repeatedly take out the smallest value in the heap, output it, then add the next value from that same array into the heap. This is called a k-way merge.

8.3 Pseudocode

Algorithm: Merge K Sorted Arrays

Input: A collection of k sorted integer arrays, Arrays

Output: Result, a single sorted array containing every element

```
1. MinHeap <- empty min-heap of (value, arrayIndex, position)
2. for each array in Arrays, with index arrayIndex do
3.     if array is not empty then
4.         Insert(MinHeap, (array[0], arrayIndex, 0))
5.     end if
6. end for
7. Result <- empty array
8. while MinHeap is not empty do
9.     (value, arrayIndex, position) <- RemoveMin(MinHeap)
10.    Append(Result, value)
11.    nextPos <- position + 1
12.    if nextPos < SizeOf(Arrays[arrayIndex]) then
13.        Insert(MinHeap, (Arrays[arrayIndex][nextPos], arrayIndex, nextPos))
14.    end if
15. end while
16. return Result
```

8.4 Correctness

At every step, right before taking a value out of the heap, the heap holds exactly one candidate per array that still has elements left, and each candidate is the smallest value that array has not yet produced (this is guaranteed because each input array is already sorted). So the true smallest value among everything remaining must be one of these candidates. Taking the heap minimum each time produces the next smallest overall value, so the Result array ends up fully sorted.

8.5 Complexity Analysis

Let N be the total number of elements across all k arrays. Lines 2–6 insert at most k first elements, each moving at most $\lfloor \log_2 k \rfloor$ levels. The loop on lines 8–15 runs exactly N times, since every element leaves the heap once. Each pass does constant work plus one sift of at most $\lfloor \log_2 k \rfloor$ levels:

$$T_{worst}(N) \leq c_1 k \log_2 k + N(c_2 + c_3 \log_2 k) = \Theta(N \log k)$$

$$T_{best}(N) = c_1 k + c_2 N$$

The best case is when no sifting is needed, for example when the arrays hold non-overlapping ranges in order. This is always at least as good as combining everything and re-sorting, which would cost $\Theta(N \log N)$. The heap takes $\Theta(k)$ extra space, and the output takes $\Theta(N)$.

Best	Average	Worst	Aux. Space
$\Theta(N + k)$	$O(N \log k)$	$\Theta(N \log k)$	$\Theta(k)$

8.6 Testing

Test	Input	Expected Output	Edge Case?
1	$\{1,4,5\}, \{1,3,4\}, \{2,6\}$	1 1 2 3 4 4 5 6	No
2	$\{\}, \{2,5\}, \{\}$	2 5	Yes (empty arrays)
3	no arrays at all	empty output	Yes ($k = 0$)

Execution screenshots for these tests are in the Appendix, Problem 8.

9 Kth Largest Element

9.1 Understand the Problem

Input: An unsorted integer array and an integer k .

Output: The value at position k if the array were sorted from largest to smallest. Duplicate values count as separate positions.

Edge Cases:

- $k = 1$ (the maximum value)
- $k = n$ (the minimum value)
- k less than 1, or k greater than n - invalid
- All duplicate values

9.2 Design the Solution

Keep a min-heap of size k . It only ever holds the k largest values seen so far, and its smallest value (the top of the heap) is always the answer once every element has been processed. This gives $O(n \log k)$, which is faster than a full sort when k is much smaller than n .

9.3 Pseudocode

Algorithm: Kth Largest Element - Min-Heap Approach

Input: An unsorted array, `Nums`, and an integer. ($k = 1$ means the largest)

Output: `KthLargest`, the value in position k if sorted from largest to smallest

```
1. if  $k \leq 0$  or  $k > \text{SizeOf}(\text{Nums})$  then
2.     report invalid input and stop
3. end if
4. MinHeap  $\leftarrow$  empty min-heap
5. for each value in Nums do
6.     if  $\text{SizeOf}(\text{MinHeap}) < k$  then
7.         Insert(MinHeap, value)
8.     else if value > PeekMin(MinHeap) then
9.         RemoveMin(MinHeap)
10.        Insert(MinHeap, value)
11.    end if
12. end for
13. return PeekMin(MinHeap)
```

9.4 Correctness

The heap always holds the k largest values seen so far. Once it is full, if a new value beats the heap's current smallest value, that smallest value can no longer be among the true k largest overall, so it is safely removed. After every element has been processed, the smallest value left in the heap is, by definition, the k th largest value overall. Duplicate values are treated as independent values, so no special handling is needed for them.

9.5 Complexity Analysis

Lines 6–7 insert the first k values, each moving at most $\lfloor \log_2 k \rfloor$ levels. Every later value costs one comparison (line 8), plus a replace and sift of at most $\lfloor \log_2 k \rfloor$ levels when it beats the heap minimum (lines 9–10):

$$T_{\text{worst}}(n) \leq c_1 n + c_2 n \log_2 k \quad T_{\text{best}}(n) = c_1 n$$

The worst case is increasing input, where every value enters the heap and moves the full height. The best case is when the first k values are the largest and arrive smallest-first, so nothing sifts and no later value enters. Space is $\Theta(k)$.

Best	Average	Worst	Aux. Space
$\Theta(n)$	$O(n \log k)$	$\Theta(n \log k)$	$\Theta(k)$

9.6 Testing

Test	Input	Expected Output	Edge Case?
1	3 2 1 5 6 4, k=2	5	No
2	7 10 4 3, k=1	10	Yes (k = 1, maximum)
3	5 5 5 2, k=4	2	Yes (k = n, duplicates)

Execution screenshots for these tests are in the Appendix, Problem 9.

10 Count Inversions

10.1 Understand the Problem

Input: An array of integers, for example [2, 4, 1, 3, 5].

Output: A 64-bit integer count of pairs (i, j) where i is less than j and values[i] is greater than values[j].

Requirement: the main solution must run in $O(n \log n)$ time, using a modified Merge Sort. A brute-force $O(n^2)$ solution may be used only to double-check the answer on small inputs.

Edge Cases:

- Empty array or single element - 0 inversions
- Already sorted array - 0 inversions (best case)
- Reverse-sorted array - the maximum possible inversions, $n(n-1)/2$
- Duplicate values - must not count as inversions

10.2 Design the Solution

Modify Merge Sort so that, during the merge step, whenever an element is taken from the right half before all remaining left-half elements have been placed, that right-half element is smaller than every one of those remaining left-half elements. Each of those forms an inversion with it, so counting these during the merge avoids ever comparing all pairs directly.

10.3 Pseudocode

Algorithm: Count Inversions - Recursive Driver

Input: An array A

Output: A sorted copy of A, and InvCount (64-bit), the number of inversions

```
1. procedure CountAndSort(A)
2.   n ← SizeOf(A)
3.   if n ≤ 1 then
4.     return (A, 0)
5.   end if
6.   Middle ← floor(n / 2)
7.   (L, LeftCount) ← CountAndSort(A[0..Middle-1])
8.   (R, RightCount) ← CountAndSort(A[Middle..n-1])
9.   return MergeAndCount(L, R, LeftCount + RightCount)
10. end procedure
```

Algorithm: MergeAndCount Subroutine

Input: Sorted arrays L and R, and the running count InvCount

Output: Merged, the sorted combination of L and R, and the updated InvCount

```
1. procedure MergeAndCount(L, R, InvCount)
2.   Merged ← empty array
3.   i ← 0
4.   j ← 0
5.   while i < SizeOf(L) and j < SizeOf(R) do
6.     if L[i] ≤ R[j] then
7.       Append(Merged, L[i])
8.       i ← i + 1
9.     else
```

```

10.         Append(Merged, R[j])
11.         InvCount <- InvCount + (SizeOf(L) - i)
12.         j <- j + 1
13.     end if
14. end while
15.     append the remaining elements of L, then of R, to Merged
16.     return (Merged, InvCount)
17. end procedure

```

10.4 Correctness

Merge Sort itself sorts correctly by standard divide-and-conquer reasoning. Every inversion falls into exactly one of three categories: both indices in the left half (counted by the left recursive call), both indices in the right half (counted by the right recursive call), or split across both halves (counted during the merge). Because elements never move between halves during the divide step, every left-half element keeps a smaller original index than every right-half element, no matter how it gets reordered inside its own half. So when $R[j]$ is chosen over the remaining elements of L (line 10), each of those remaining elements has both a smaller original index and a larger value, which is exactly the definition of an inversion. Using less-than-or-equal (not strictly less-than) when comparing $L[i]$ and $R[j]$ on line 6 makes sure equal values are never counted as inversions.

10.5 Complexity Analysis

Each call splits the array in half (lines 7–8), and MergeAndCount places all n elements exactly once at constant cost each:

$$T(n) = 2T(n/2) + cn, \quad T(1) = c$$

Expanding i times gives $T(n) = 2^i T(n/2^i) + icn$. Stopping at $i = \log_2 n$ gives

$$T(n) = cn + cn \log_2 n = \Theta(n \log n)$$

The splitting and merging do not depend on the values, so best, average, and worst case match. The brute-force check makes $n(n-1)/2$ comparisons, which is $\Theta(n^2)$, so it is used only to validate. The halves copied along the current recursion path total at most $2n$, plus $\log_2 n$ stack frames, so auxiliary space is $\Theta(n)$.

Best	Average	Worst	Aux. Space
$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$

The running count must be stored as a 64-bit integer, since the maximum possible count, $n(n-1)/2$, can be larger than a normal 32-bit integer can hold for large n .

10.6 Testing

Test	Input	Expected Output	Edge Case?
1	2 4 1 3 5	3	No (matches assignment example)
2	1 2 3 4 5	0	Yes (already sorted, best case)
3	5 4 3 2 1	10	Yes (reverse sorted, worst case)

Execution screenshots for these tests, along with the brute-force result used to double-check Test 1, are in the Appendix, Problem 10.

11 Appendix: Screenshots

11.1 Problem 1: Merge Overlapping Intervals

```
EECE 7205 Assignment 01
1. Merge Overlapping Intervals
2. Sort Array of 0s, 1s, and 2s
3. Largest Number from Concatenation
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points to Origin
7. Top K Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions
0. Exit
Choose a problem: 1

Problem 1: Merge Overlapping Intervals
Test 1 input:  [1,3] [2,6] [8,10] [15,18]
Test 1 output: [1,6] [8,10] [15,18]

Test 2 input:  [1,4] [4,5]
Test 2 output: [1,5]

Test 3 input:  (empty)
Test 3 output: (empty)
```

11.2 Problem 2: Sort Array of 0s, 1s, 2s

EECE 7205 Assignment 01

1. Merge Overlapping Intervals
2. Sort Array of 0s, 1s, and 2s
3. Largest Number from Concatenation
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points to Origin
7. Top K Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions
0. Exit

Choose a problem: 2

Problem 2: Sort Array of 0s, 1s, and 2s

Test 1 input: 2 0 2 1 1 0

Test 1 counting: 0 0 1 1 2 2

Test 1 dutch flag: 0 0 1 1 2 2

Test 2 input: 0 0 1 1 2 2

Test 2 counting: 0 0 1 1 2 2

Test 2 dutch flag: 0 0 1 1 2 2

Test 3 input: (empty)

Test 3 counting: (empty)

Test 3 dutch flag: (empty)

Test 4 input: 1 1 1 1

Test 4 counting: 1 1 1 1

Test 4 dutch flag: 1 1 1 1

11.3 Problem 3: Largest Number from Concatenation

EECE 7205 Assignment 01

1. Merge Overlapping Intervals
2. Sort Array of 0s, 1s, and 2s
3. Largest Number from Concatenation
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points to Origin
7. Top K Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions
0. Exit

Choose a problem: 3

Problem 3: Largest Number from Concatenation

Test 1 input: 3 30 34 5 9

Test 1 output: 9534330

Test 2 input: 0 0

Test 2 output: 0

Test 3 input: 5

Test 3 output: 5

11.4 Problem 4: Group Anagrams

EECE 7205 Assignment 01

1. Merge Overlapping Intervals
2. Sort Array of 0s, 1s, and 2s
3. Largest Number from Concatenation
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points to Origin
7. Top K Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions
0. Exit

Choose a problem: 4

Problem 4: Group Anagrams

Test 1 input: "eat" "tea" "tan" "ate" "nat" "bat"

Test 1 output: {"eat" "tea" "ate"} {"tan" "nat"} {"bat"}

Test 2 input: "" ""

Test 2 output: {"" ""}

Test 3 input: "abc" "def" "ghi"

Test 3 output: {"abc"} {"def"} {"ghi"}

11.5 Problem 5: Meeting Rooms

EECE 7205 Assignment 01

1. Merge Overlapping Intervals
2. Sort Array of 0s, 1s, and 2s
3. Largest Number from Concatenation
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points to Origin
7. Top K Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions
0. Exit

Choose a problem: 5

Problem 5: Meeting Rooms

Test 1 input: [0,30] [5,10] [15,20]

Test 1 output: false

Test 2 input: [1,4] [4,5]

Test 2 output: true

Test 3 input: (empty)

Test 3 output: true

11.6 Problem 6: K Closest Points to Origin

EECE 7205 Assignment 01

1. Merge Overlapping Intervals
2. Sort Array of 0s, 1s, and 2s
3. Largest Number from Concatenation
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points to Origin
7. Top K Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions
0. Exit

Choose a problem: 6

Problem 6: K Closest Points to Origin

Test 1 input: (1,3) (-2,2) (5,8) (0,1) k = 2

Test 1 sorting: (0,1) (-2,2)

Test 1 heap: (-2,2) (0,1)

Test 2 input: (1,1) (2,2) k = 2

Test 2 sorting: (1,1) (2,2)

Test 2 heap: (2,2) (1,1)

Test 3 input: (1,1) (2,2) k = 0

Test 3 sorting: (empty)

Test 3 heap: (empty)

11.7 Problem 7: Top K Frequent Elements

EECE 7205 Assignment 01

1. Merge Overlapping Intervals
2. Sort Array of 0s, 1s, and 2s
3. Largest Number from Concatenation
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points to Origin
7. Top K Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions
0. Exit

Choose a problem: 7

Problem 7: Top K Frequent Elements

Test 1 input: 1 1 1 2 2 3 k = 2

Test 1 output: 1 2

Test 2 input: 3 1 2 k = 2

Test 2 output: 1 2

Test 3 input: 5 5 6 k = 2

Test 3 output: 5 6

11.8 Problem 8: Merge K Sorted Arrays

EECE 7205 Assignment 01

1. Merge Overlapping Intervals
2. Sort Array of 0s, 1s, and 2s
3. Largest Number from Concatenation
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points to Origin
7. Top K Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions
0. Exit

Choose a problem: 8

Problem 8: Merge K Sorted Arrays

Test 1 input:

array 1: 1 4 5

array 2: 1 3 4

array 3: 2 6

Test 1 output: 1 1 2 3 4 4 5 6

Test 2 input:

array 1: (empty)

array 2: 2 5

array 3: (empty)

Test 2 output: 2 5

Test 3 input:

no arrays

Test 3 output: (empty)

11.9 Problem 9: Kth Largest Element

EECE 7205 Assignment 01

1. Merge Overlapping Intervals
2. Sort Array of 0s, 1s, and 2s
3. Largest Number from Concatenation
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points to Origin
7. Top K Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions
0. Exit

Choose a problem: 9

Problem 9: Kth Largest Element

Test 1 input: 3 2 1 5 6 4 k = 2

Test 1 output: 5

Test 2 input: 7 10 4 3 k = 1

Test 2 output: 10

Test 3 input: 5 5 5 2 k = 4

Test 3 output: 2

11.10 Problem 10: Count Inversions

EECE 7205 Assignment 01

1. Merge Overlapping Intervals
2. Sort Array of 0s, 1s, and 2s
3. Largest Number from Concatenation
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points to Origin
7. Top K Frequent Elements
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions
0. Exit

Choose a problem: 10

Problem 10: Count Inversions

Test 1 input: 2 4 1 3 5

Test 1 inversions: 3

Test 1 brute force: 3

Test 2 input: 1 2 3 4 5

Test 2 inversions: 0

Test 2 brute force: 0

Test 3 input: 5 4 3 2 1

Test 3 inversions: 10

Test 3 brute force: 10